Beyond Compilation: An Empirical Evaluation of Functional Correctness and Static Quality in LLM-Generated TypeScript Code

Yehezkiel Gunawan
Bina Nusantara University
Jakarta, Indonesia
yehezkiel.gunawan@binus.ac.id

***Abstract*—Compilation is often used as an initial check on code produced by large language models, but it does not establish functional correctness or adherence to static-quality rules. This study evaluates those properties separately for TypeScript. We constructed 15 deterministic utility tasks in five software-engineering categories and generated one solution per task with each of three models, producing 45 programs. Every program passed through source extraction and safety inspection, strict TypeScript compilation, task-specific Vitest tests, and ESLint analysis. Of the 45 programs, 37 compiled, 36 passed all visible tests, and 34 passed the complete quality gate. Eight compilation failures arose from possibly undefined indexed access under strict TypeScript settings. Among the 37 compilable programs, one failed a functional test. Two programs passed all tests but violated selected ESLint rules. The results show modest but measurable attrition after compilation and demonstrate that the three checks answer different questions. For the evaluated tasks and models, generated TypeScript should therefore be treated as candidate code that remains subject to the compiler, tests, and static analysis before integration.**

***Index Terms*—large language models, code generation, TypeScript, functional correctness, static analysis, empirical software engineering**

I. Introduction

Large language models (LLMs) now produce source code from natural-language requirements, but plausible source text is not equivalent to validated software. Early code-generation benchmarks made execution-based testing central to evaluation. HumanEval, for example, measures whether generated functions pass task tests and introduced pass@k for repeated samples [1]. Subsequent empirical work has examined generated code across languages, tools, and quality dimensions [2]–[4]. These studies provide a stronger basis than judging code by syntax or readability alone.

TypeScript makes the distinction between plausibility and validity particularly visible. Its compiler can reject code that would execute in JavaScript because strict options retain uncertainty about indexed values, optional properties, or generic types. A successful compilation still establishes neither compliance with behavioral requirements nor conformance to project-level quality rules. The relevant outcomes form sequential, non-equivalent checks: type-level validity, test-suite correctness, and selected static quality.

Prominent function-level evaluations remain strongly associated with algorithmic tasks, while newer benchmarks extend toward class-level generation and repository issue resolution [5], [6]. Compact TypeScript utilities occupy a useful middle ground. They are deterministic and independently testable, yet reflect validation, URL handling, asynchronous control, caching, rate limiting, and pagination found in web development.

This study asks how reliably contemporary LLMs produce TypeScript that survives three conventional development checks. We address the following questions:

RQ1: To what extent does generated TypeScript compile under strict compiler settings?

RQ2: To what extent are compilable programs functionally correct under task-specific tests?

RQ3: What selected static-quality issues remain, including in programs that pass all tests?

The contribution is a reproducible 15-task TypeScript corpus, a controlled 45-generation comparison under one prompt, and an empirical account of attrition across compilation, testing, and ESLint. The artifact, including tasks, prompts, raw generations, evaluation records, and report scripts, is available online.[1]

II. Related Work

A. Functional evaluation of generated code

HumanEval established execution against unit tests as a standard measure for function synthesis [1]. Xu et al. found substantial variation across code models and programming languages [2]. Nguyen and Nadi evaluated Copilot suggestions in multiple languages using tests and complexity measures [3], while Dakhel et al. compared generated and human solutions and documented correctness and reproducibility limitations [7]. Test quality also affects reported performance. EvalPlus expanded HumanEval tests and exposed incorrect programs accepted by the original suites [8]. Our use of visible deterministic tests therefore measures conformance to those suites, not proof of correctness.

Recent benchmarks broaden what constitutes a programming task. ClassEval targets class-level generation [5]; SWE-

[1] https://github.com/yehezkielgunawan/icdees-2026

TABLE I
TYPESCRIPT TASK SET

Category	No.	Representative task
Data transformation	3	Group records by a derived key
Validation	3	Validate a registration form
String and URL processing	3	Convert a title to a URL-safe slug
Async and Promise utilities	3	Retry an asynchronous operation
Web backend utilities	3	Implement an in-memory TTL cache

bench evaluates changes for issues in existing repositories [6]. Our tasks are smaller by design. They isolate strict TypeScript behavior in common utilities and support a controlled comparison, at the cost of lower application-level realism.

B. Quality, robustness, and use

Generated-code studies have measured properties beyond tests. Yetiştiren et al. assessed multiple quality dimensions [4], and Al Madi examined readability and complexity [9]. Security studies demonstrate that functional appearance does not cover other software properties, although our ESLint rules do not constitute security analysis [10]. Barke et al. found that programmers alternate between acceleration and exploration when using generated suggestions, both of which require inspection and revision [11].

Generation conditions also constrain reproducibility. Semantically similar prompts can change Copilot outputs [12]; prompt language and repeated runs can change suggestion outcomes [13]. Ouyang et al. report nondeterminism in ChatGPT code generation [14]. These findings motivate our fixed prompt and precise artifact capture, while also limiting what can be inferred from one generation per model-task pair. A recent systematic review confirms the breadth of LLM uses and evaluation practices in software engineering [15]. Against that broader literature, this study contributes ecosystem-specific evidence rather than a new general-purpose benchmark.

III. METHOD

A. Research design and tasks

The independent variable was the model. Dependent variables were strict compilation, full test-suite success, selected ESLint findings, and passage through all gates. The task, prompt, execution order, test suite, compiler, lint rules, and dependency lock were held constant across models.

The corpus contains 15 tasks divided evenly among five categories (Table I). Selection required deterministic behavior, no external dependency, bounded source size, and relevance to TypeScript or web-backend development. Examples include building a category tree, validating pagination parameters, normalizing URL paths, limiting asynchronous concurrency, and implementing cursor pagination. Each task has a required exported signature, a reference implementation, and four to six Vitest cases. The corpus contains 75 tests in total. Reference implementations passed all tests, strict compilation, and ESLint under the same evaluator.

B. Generation procedure

We evaluated three identifiers from the OpenAI provider: `gpt-5.6-sol`, `gpt-5.6-terra`, and `gpt-5.6-luna`, labeled Sol, Terra, and Luna. These identifiers were active in the recorded provider snapshot but do not identify immutable public model revisions. Each model generated one program for each task, for 45 generations. The campaign ran on September 3, 2026 through OpenCode 1.18.26.

One prompt template required source code only, the exact signature, no imports or external dependencies, no `any`, no TypeScript suppression comments, and no process, file-system, network, or global mutable-state access. The benchmark agent could not call tools and was limited to one step. A temperature of zero was requested, but the runtime marked temperature unsupported for all three models and recorded no applied value. The configured 4096-token output limit was recorded but not transmitted by the generation call, so it is not treated as an enforced control. All raw responses, extracted sources, hashes, and metadata were retained. All 45 generations and extractions succeeded without normalization changes.

C. Evaluation pipeline

The evaluator first checked for empty output and disallowed operations. It then compiled each candidate and its tests with TypeScript 5.8.2 using `strict`, `noImplicitAny`, `noUncheckedIndexedAccess`, and `exactOptionalPropertyTypes`, among other module-isolation settings. Compile failure stopped later stages. Compilable candidates proceeded to their Vitest 3.2.4 suite and ESLint 9.35.0 in parallel.

A candidate was *fully correct* when at least one test ran and every test passed. A candidate was *static-clean* when ESLint completed with zero errors. Selected rules included a cyclomatic-complexity maximum of 12, `prefer-const`, equality and control-flow rules, and TypeScript rules for explicit `any` and unused variables. The *full gate* required successful extraction, compilation, full correctness, and zero ESLint errors. Compilation, functional, and full-gate rates use all generations as their denominator. Static-clean and compiled-but-incorrect rates use compilable programs; correct-but-static-issue rates use fully correct programs. Given 15 paired tasks and one sample per cell, we report counts and percentages without hypothesis tests or model-ranking claims.

IV. RESULTS

Table II reports each model and the aggregate outcomes. Fig. 1 shows the same sequential gates to make attrition explicit.

A. RQ1: Strict compilation

Thirty-seven of 45 candidates compiled, an aggregate rate of 82.2%. The observed model rates ranged from 12/15 (80.0%) for Sol and Terra to 13/15 (86.7%) for Luna. Every rejected candidate had exactly one compiler error, and all eight errors involved possibly undefined indexed access under `noUncheckedIndexedAccess`.

TABLE II
RESULTS BY MODEL. COMPILATION, FULL CORRECTNESS, AND FULL GATE USE GENERATED CANDIDATES AS DENOMINATORS. STATIC-CLEAN AND COMPILED-INCORRECT USE COMPILED CANDIDATES; CORRECT-STATIC-ISSUE USES FULLY CORRECT CANDIDATES.

Model	Compiled	Fully correct	Static-clean	Full gate	Compiled, incorrect	Correct, static issue
GPT-5.6 Sol	12/15 (80.0%)	12/15 (80.0%)	12/12 (100.0%)	12/15 (80.0%)	0/12 (0.0%)	0/12 (0.0%)
GPT-5.6 Terra	12/15 (80.0%)	11/15 (73.3%)	11/12 (91.7%)	10/15 (66.7%)	1/12 (8.3%)	1/11 (9.1%)
GPT-5.6 Luna	13/15 (86.7%)	13/15 (86.7%)	12/13 (92.3%)	12/15 (80.0%)	0/13 (0.0%)	1/13 (7.7%)
All	**37/45 (82.2%)**	**36/45 (80.0%)**	**35/37 (94.6%)**	**34/45 (75.6%)**	**1/37 (2.7%)**	**2/36 (5.6%)**

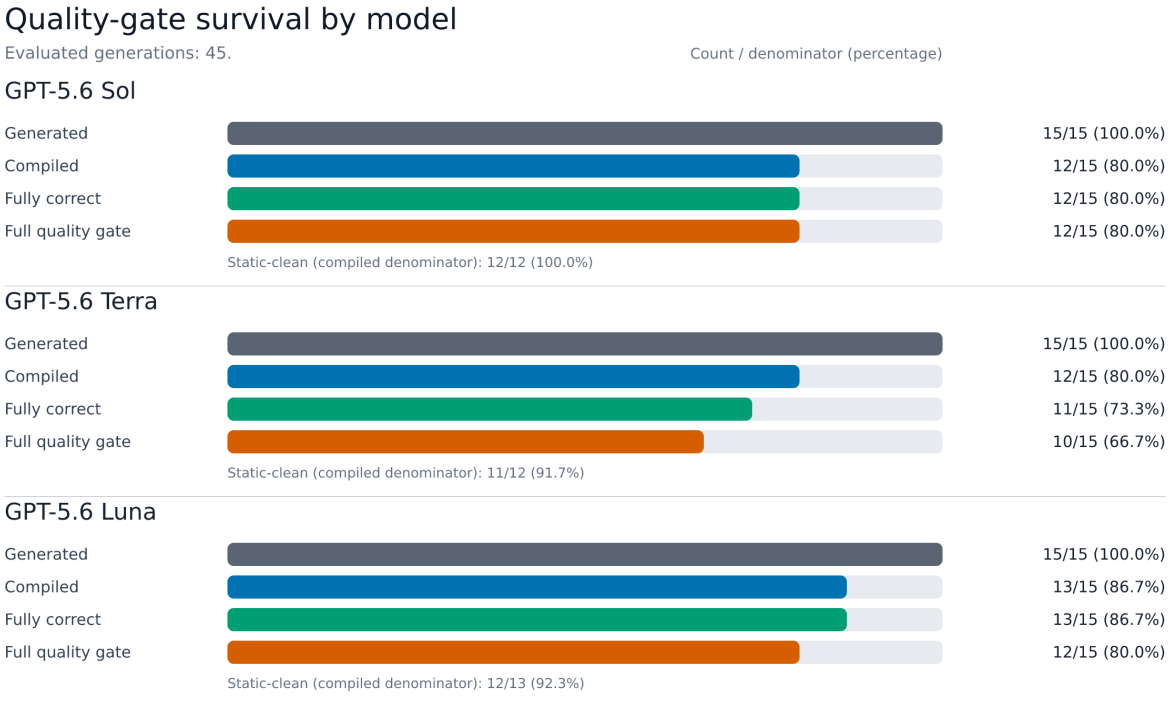

Fig. 1. Quality-gate survival for the 45 primary generations. Static-clean rates are shown separately because their denominator is the number of compilable candidates.

The failures were concentrated in three tasks rather than spread across the corpus. Sol and Terra failed task 08 by constructing an array whose inferred element type remained `string | undefined`. All three models failed task 11 when passing `items[index]`, typed as `T | undefined`, to a worker expecting `T`. All three failed task 15 through a similar access to the last page item. These are strict type failures, not observed test failures: the pipeline did not run Vitest or ESLint after compilation failed.

B. RQ2: Functional correctness

Among 37 compilable candidates, 36 passed their complete task suite (97.3%). Relative to all generated candidates, the functional success rate was 36/45 (80.0%). Across compiled programs, 178 of 179 executed test cases passed, but candidate-level outcomes remain the unit of analysis.

The sole compiled-but-incorrect candidate was Terra's registration validator in task 04. It passed four of five tests. For an empty object, it reported missing username, email, and password fields but omitted `confirmPassword`, because the implementation compared two absent values as equal. This case directly separates compilation from behavioral compliance, although its frequency, 1/37, does not support a claim that compiled programs often failed the tests.

C. RQ3: Static quality and the full gate

Thirty-five of 37 compiled candidates were static-clean (94.6%). Both lint failures had passed all functional tests. Luna's URL-path normalizer had cyclomatic complexity 13 against the configured maximum of 12. Terra's Promise-timeout utility declared a reassignment-free timer with `let`, violating `prefer-const`. No linted candidate used explicit `any`, suppression comments, or generated a warning.

The static stage reduced the fully correct set from 36 to 34 full-gate programs. Sol and Luna each passed the complete gate for 12/15 candidates (80.0%); Terra passed for 10/15 (66.7%). These differences are descriptive observations from one run, not estimates of stable model ordering.

V. Discussion

A. What compilation established

Compilation removed eight programs before behavioral evaluation and exposed one recurring TypeScript-specific weakness: candidates relied on runtime bounds or existence checks that did not narrow general indexed access enough for the declared type. The reference solutions needed explicit narrowing or a justified non-null assertion. This result is relevant to TypeScript development because generated JavaScript logic can appear executable while failing a project's strict type contract.

Compilation was nevertheless a strong screen in this sample. Only one of 37 compilable programs failed a visible test. The data therefore support a narrow conclusion: compilation and functional correctness are distinct, but the observed post-compilation functional attrition was small. That distinction matters when the cost of one missed boundary condition is high, yet it should not be inflated into a broad estimate of LLM unreliability.

B. Tests and static analysis answer different questions

The registration example shows how a short condition can mishandle two simultaneously missing values while remaining type-correct. Tests supplied evidence about that requirement. Their evidence is finite, however. EvalPlus has shown that expanded suites can expose code accepted by smaller suites [8]; our label "fully correct" means success on four to six visible tests, not specification proof.

The two ESLint findings did not indicate functional defects. One captured a local declaration choice; the other marked complexity just above a project threshold. This modest attrition is still informative because it demonstrates that passing tests does not imply compliance with selected project rules. It does not support claims about maintainability, security, or production readiness. Those constructs require broader measures and, in some cases, human assessment.

C. Implications

For developers, the sequential pipeline offers a simple policy: treat generated source as a candidate patch and apply the same compiler, tests, and static checks used for human-written code. The stages are complementary. Strict compilation catches type-contract violations cheaply; tests evaluate specified behavior; static analysis enforces selected project conventions. For researchers, explicit denominators are necessary because compile failures never reach later stages. Reporting a static-clean rate over all generations would conflate rejection with unobserved lint behavior.

VI. Scope, Limitations, and Threats to Validity

This study covers 45 single-attempt generations across 15 isolated TypeScript utility tasks and three provider model identifiers. It measures strict compilation, performance on visible task-specific tests, and compliance with selected ESLint rules. The findings characterize this controlled campaign; they do not establish general LLM coding performance, production readiness, or stable differences among models.

Construct validity. Unit tests approximate functional correctness and cover only four to six cases per task. One task description included URL query and fragment handling that its tests did not exercise, which further limits the "fully correct" label. ESLint represents a small set of static rules, not general maintainability or security. Strict compilation failures mainly measure explicit handling of conservative indexed-access types and should not be interpreted as algorithmic errors.

Internal and conclusion validity. Each model-task pair has one generation. Because temperature control was unsupported, the study cannot separate stable model behavior from sampling variation. Fixed prompts and complete raw-output capture improve auditability but do not remove nondeterminism or provider updates [12], [14]. The small paired sample precludes strong ranking claims. The manifest froze tasks, prompt, study configuration, and a dependency lockfile, but did not bind the evaluator source revision or benchmark-agent configuration. The exact campaign-time Node version, operating system, and provider-side model revisions were not captured.

External validity. The corpus contains 15 isolated TypeScript utilities, three provider identifiers, one prompt, and visible tests. Results do not generalize to full applications, interactive assistants, repository agents, other languages, or other model revisions. The compact design improves control and reproducibility but is less realistic than class-level or repository-level benchmarks.

VII. Conclusion

In 45 generated TypeScript programs, 37 compiled, 36 passed all visible tests, and 34 passed compilation, tests, and selected ESLint rules. The small losses after compilation were enough to show that the gates are not interchangeable. Within this task set, strict compilation was an effective first screen, not a complete quality judgment. Generated TypeScript should remain subject to ordinary software-engineering validation before integration.

Acknowledgment

The author used OpenCode with GPT-5.6-sol and GPT-5.6-luna primarily for technical coding tasks, support for the systematic literature review through Exa AI MCP—including source discovery, metadata verification, and source organization—and language editing, including paraphrasing and correction of language errors. The author independently reviewed and validated all AI-assisted outputs, verified the empirical claims and cited sources, and assumes full responsibility for the accuracy, interpretation, and final content of the manuscript.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis,

E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[2] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. ACM, 2022, pp. 1–10. [Online]. Available: https://doi.org/10.1145/3520312.3534862

[3] N. Nguyen and S. Nadi, "An empirical evaluation of GitHub Copilot's code suggestions," in *Proceedings of the 19th International Conference on Mining Software Repositories*. ACM, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1145/3524842.3528470

[4] B. Yetistiren, I. Ozsoy, and E. Tuzun, "Assessing the quality of GitHub Copilot's code generation," in *Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering*. ACM, 2022, pp. 62–71. [Online]. Available: https://doi.org/10.1145/3558489.3559072

[5] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–13. [Online]. Available: https://doi.org/10.1145/3597503.3639219

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations*, 2024, pp. 54 107–54 157. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[7] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "GitHub Copilot AI pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, p. 111734, 2023. [Online]. Available: https://doi.org/10.1016/j.jss.2023.111734

[8] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 21 558–21 572. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html

[9] N. Al Madi, "How readable is model-generated code? examining readability and visual inspection of GitHub Copilot," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1145/3551349.3560438

[10] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *2022 IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 754–768. [Online]. Available: https://doi.org/10.1109/SP46214.2022.9833571

[11] S. Barke, M. B. James, and N. Polikarpova, "Grounded Copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 85–111, 2023, art. no. 78. [Online]. Available: https://doi.org/10.1145/3586030

[12] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, "On the robustness of code generation techniques: An empirical study on GitHub Copilot," in *2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 2023, pp. 2149–2160. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00181

[13] K. Koyanagi, D. Wang, K. Noguchi, M. Kondo, A. Serebrenik, Y. Kamei, and N. Ubayashi, "Exploring the effect of multiple natural languages on code suggestion using GitHub Copilot," in *Proceedings of the 21st International Conference on Mining Software Repositories*. ACM, 2024, pp. 481–486. [Online]. Available: https://doi.org/10.1145/3643991.3644917

[14] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–28, 2025, art. no. 42. [Online]. Available: https://doi.org/10.1145/3697010

[15] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024, art. no. 220. [Online]. Available: https://doi.org/10.1145/3695988